

Module 3: Spring Data JPA, Hibernate, and Database Concepts

Questions

Q1. What is an ORM, and what does Hibernate do?

Answer: ORM stands for Object-Relational Mapping. It maps Java objects to database tables so developers can work with objects instead of writing raw SQL. Hibernate is an ORM implementation for Java. In a banking app, instead of writing JDBC to insert an account row, you create an `Account` entity object and call `repository.save(account)`. Hibernate translates that into an INSERT statement. Remember that Hibernate is one JPA provider; EclipseLink and OpenJPA are others, and switching providers changes provider-specific tuning but keeps the annotation model.

Q2. What is the difference between JPA and Hibernate?

Answer: JPA is a specification that defines how to map Java objects to relational tables. Hibernate is the most popular implementation of JPA. Spring Data JPA sits on top of Hibernate and generates repository implementations for you. When a query misbehaves, you usually need to inspect Hibernate SQL logs or `SessionFactory` settings, not just the Spring Data method name.

Q3. What happens internally when you call `repository.save(entity)`?

Answer: Spring Data JPA routes the call to `SimpleJpaRepository.save`. It checks the entity ID. If the ID is null or the version indicates a new instance, it calls `entityManager.persist(entity)`. Otherwise it calls `merge(entity)`. The entity becomes associated with the current persistence context. The actual INSERT/UPDATE is usually deferred until flush time, typically at transaction commit. If the JSON payload already contains an ID, Hibernate may perform a merge and update an existing row instead of inserting.

Q4. What is the difference between `persist()` and `merge()`?

Answer: `persist()` takes a transient entity and makes it managed. It should not have an existing ID. `merge()` copies the state of a detached entity into a managed instance, inserting if needed. `persist()` returns the same instance; `merge()` returns the managed copy. Calling `merge()` with a partially populated detached entity can overwrite non-null database fields with nulls, so partial updates should fetch the managed entity first.

Q5. What is an Entity lifecycle in Hibernate?

Answer: Entities move through four states: transient (new object, no ID, unknown to Hibernate), persistent (associated with the EntityManager, changes tracked and flushed at commit), detached (was persistent but session closed, changes not tracked), and removed (marked for deletion, row deleted at flush). A controller that maps JSON to an entity and calls `save()` may accidentally update an existing row if the JSON contains an ID.

Q6. What is the persistence context, and how does it act as a first-level cache?

Answer: The persistence context is the EntityManager's in-memory cache of managed entities. Within a single session, loading the same entity by ID twice returns the same Java instance, and dirty checking happens against this cache at flush time. Reading an entity, modifying it, and reading it again in the same transaction shows the in-memory change before flush, but another process updating the database between reads can cause stale data because the cache is isolated to one session.

Q7. What is the difference between CrudRepository, PagingAndSortingRepository, and JpaRepository?

Answer: CrudRepository provides basic CRUD operations. PagingAndSortingRepository adds pagination and sorting support. JpaRepository adds bulk operations (`saveAll`, `deleteInBatch`), `saveAndFlush`, `flush`, `getReferenceById`, and Query By Example. Use JpaRepository for new Spring Boot code and CrudRepository only when you want to avoid coupling to JPA-specific features.

Q8. What is FetchType.LAZY vs FetchType.EAGER?

Answer: LAZY loads related data only when it is accessed. EAGER loads it immediately with the parent entity. Eager loading on broad object graphs causes large SQL joins, Cartesian products, memory bloat, and unpredictable performance. Lazy loading keeps the initial query small but requires explicit loading inside a transaction to avoid `LazyInitializationException`.

Q9. What is the N+1 SELECT problem?

Answer: It happens when one query fetches N parent rows, and then accessing a lazy collection on each parent triggers N additional SELECT statements. Fix it with `JOIN FETCH`, entity graphs, batch fetching, DTO projections, or subselect fetching. A customer list endpoint that returns 100 orders and lazily loads each order's items outside the service can make 101 queries instead of one, causing high database CPU under load.

Q10. What is `LazyInitializationException`, and what are the correct fixes?

Answer: It occurs when code accesses a lazy association after the `EntityManager` has closed. In Spring web apps this often happens during JSON serialization because the session was bound to the now-completed transaction. Fix it by loading all needed data inside the transaction using `JOIN FETCH` or an entity graph, or by returning DTO projections. Do not enable `Open Session in View` as a default fix because it hides the problem and causes N+1 issues in production.

Q11. What is the difference between `findById()` and `getReferenceById()`?

Answer: `findById()` hits the database immediately and returns an `Optional` with the full entity. `getReferenceById()` returns a Hibernate proxy without a database round trip and is useful when you only need a foreign-key reference. If the referenced row does not exist, the proxy throws `EntityNotFoundException` only when a property is accessed.

Q12. Explain `mappedBy` and `@JoinColumn` with an example.

Answer: In a bidirectional relationship, only one side owns the foreign key. The owning side uses `@JoinColumn`; the inverse side uses `mappedBy`.

```
@Entity
public class Order {
    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL, orphanRemoval =
true)
    private List<OrderItem> items = new ArrayList<>();
}

@Entity
public class OrderItem {
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "order_id")
    private Order order;
}
```

Forgetting `mappedBy` or putting `@JoinColumn` on both sides creates a duplicate foreign-key column or a join table, leading to inconsistent data.

Q13. What are cascade types and `orphanRemoval`?

Answer: Cascade types propagate operations from a parent to its children: `PERSIST`, `MERGE`, `REMOVE`, `REFRESH`, `DETACH`, and `ALL`. `orphanRemoval = true` deletes a child entity when it is removed from the parent collection, even if the parent still exists. `CascadeType.ALL` on broad or shared relationships is dangerous because deleting an order line could cascade to a shared product catalog.

Q14. What is `@Transactional`, and what does it do under the hood?

Answer: Spring creates a proxy around the annotated bean. When the method is called, the proxy starts or joins a transaction, binds an `EntityManager` to it, executes the method, and commits or rolls back based on the outcome. By default, rollback happens for unchecked exceptions and `Error`, not for checked exceptions. A `@Transactional` annotation on a `private`, `final`, or `static` method is ignored because Spring cannot proxy it, and calling the method from within the same class bypasses the proxy.

Q15. What are transaction propagation levels?

Answer: `REQUIRED` joins the existing transaction or creates a new one and is the default. `REQUIRES_NEW` suspends the current transaction and starts a new one, useful for audit logging that must commit even if the main transaction rolls back. `MANDATORY` requires an existing transaction and throws if none exists. `SUPPORTS`, `NOT_SUPPORTED`, and `NEVER` are rarely used and control whether the method participates in an existing transaction. `REQUIRES_NEW` still uses the same physical database connection unless configured otherwise.

Q16. What does `@Transactional(readOnly = true)` do?

Answer: It tells Spring and Hibernate that the transaction only reads. Hibernate can skip dirty checking and avoid unnecessary flushes, and some connection pools or replication drivers may route read-only transactions to read replicas. It does not prevent writes at the database level; a modifying native query inside a read-only transaction will still execute.

Q17. What is the difference between optimistic and pessimistic locking?

Answer: Optimistic locking uses a `@Version` column. Each update checks that the version has not changed. If another transaction modified the row, the update fails with `OptimisticLockException`. Pessimistic locking uses database locks (`PESSIMISTIC_READ` / `PESSIMISTIC_WRITE`) to serialize access. Optimistic locking is better for read-heavy workloads with occasional contention; pessimistic locking is better when contention is high and you want to serialize access rather than retry.

Q18. What are the ACID properties?

Answer: Atomicity means a transaction completes entirely or not at all. Consistency means the database remains in a valid state after the transaction. Isolation means concurrent transactions do not interfere with each other. Durability means committed changes survive failures. A fund transfer must debit one account and credit another atomically, otherwise the books will be out of balance.

Q19. What are SQL isolation levels?

Answer: READ UNCOMMITTED allows dirty reads. READ COMMITTED prevents dirty reads. REPEATABLE READ prevents dirty and non-repeatable reads. SERIALIZABLE prevents all anomalies, including phantom reads. Spring/Hibernate usually follows the database default, such as READ COMMITTED in Oracle and PostgreSQL, and REPEATABLE READ in MySQL InnoDB.

Q20. What is the difference between WHERE and HAVING?

Answer: WHERE filters rows before grouping. HAVING filters groups after aggregation. You cannot use aggregate functions in WHERE except inside a subquery.

Q21. What are the differences between DDL, DML, DCL, and TCL?

Answer: DDL (Data Definition Language) includes CREATE, ALTER, DROP, and TRUNCATE. DML (Data Manipulation Language) includes SELECT, INSERT, UPDATE, and DELETE. DCL (Data Control Language) includes GRANT and REVOKE. TCL (Transaction Control Language) includes COMMIT, ROLLBACK, and SAVEPOINT.

Q22. Explain INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL JOIN.

Answer: INNER JOIN returns rows that have matches in both tables. LEFT JOIN returns all rows from the left table and matching rows from the right, filling with nulls where there is no match. RIGHT JOIN is the reverse of LEFT JOIN. FULL JOIN returns rows when there is a match in either table. A report that lists all customers and their orders should use LEFT JOIN; using INNER JOIN silently drops customers who have not placed an order.

Q23. What is the difference between UNION and UNION ALL?

Answer: UNION removes duplicates and is slower. UNION ALL keeps duplicates and is faster. Use UNION ALL when the sources are already disjoint or duplicates are acceptable.

Q24. What is normalization vs denormalization?

Answer: Normalization reduces redundancy and improves data integrity by organizing data into related tables. Denormalization intentionally adds redundancy to improve read performance. Denormalize for read-heavy reporting, pre-computed aggregates, and low-write / high-read workloads, but you must have a strategy to keep redundant data consistent.

Q25. What is an index, and how does it help?

Answer: An index is a data structure, typically a B-tree, that speeds up data retrieval by letting the

database find rows without scanning the entire table. Indexes speed up reads but slow down writes and consume disk space.

Q26. What is a composite index?

Answer: A composite index is an index on multiple columns. Column order matters. A composite index on (`status`, `created_date`) helps a query filtering by both columns, but may not help a query that filters only on `created_date`.

Q27. What is a covering index?

Answer: A covering index contains all the columns needed by a query. The database can answer the query entirely from the index without looking up the table heap, reducing I/O significantly. It helps read-heavy queries with stable access patterns.

Q28. What is the difference between a primary key and a unique key?

Answer: A primary key uniquely identifies a row, cannot be null, and there is only one per table. A unique key also enforces uniqueness but allows one null per column in most databases, and a table can have multiple unique keys.

Q29. What is a foreign key?

Answer: A foreign key enforces referential integrity between tables. It prevents orphaned rows and can cascade deletes or updates. The cost is additional index maintenance and potential locking overhead during writes.

Q30. What are stored procedures and triggers?

Answer: A stored procedure is a precompiled set of SQL statements stored in the database. A trigger automatically executes in response to an INSERT, UPDATE, or DELETE event. Triggers can make debugging hard because business logic is hidden from the application code. Stored procedures can centralize complex logic but reduce portability.

Q31. What is the difference between DELETE, TRUNCATE, and DROP?

Answer: DELETE removes specific rows, logs each row, can be rolled back in a transaction, and fires triggers. TRUNCATE removes all rows quickly with minimal logging, generally cannot be rolled back, and resets auto-increment counters. DROP removes the entire table structure, data, and dependencies. Prefer DELETE with a WHERE clause for controlled removal in production; TRUNCATE and DROP require change-management scrutiny.

Q32. What are aggregate functions in SQL?

Answer: Aggregate functions perform calculations on a set of values. Common examples are COUNT, SUM, AVG, MIN, and MAX. They are usually used with GROUP BY.

Q33. What is the difference between GROUP BY and ORDER BY?

Answer: GROUP BY groups rows with the same values into summary rows. ORDER BY sorts the result set. You can use them together.

Q34. What are window functions?

Answer: Window functions perform calculations across a set of rows related to the current row without collapsing the result set. Examples are ROW_NUMBER(), RANK(), LEAD(), LAG(), and SUM() OVER. To find the most recent transaction per account:

```
SELECT account_number, amount, transaction_date
FROM (
    SELECT account_number, amount, transaction_date,
           ROW_NUMBER() OVER (PARTITION BY account_number ORDER BY
transaction_date DESC) rn
    FROM transactions
) t
WHERE rn = 1;
```

Q35. What is a CTE?

Answer: A Common Table Expression (WITH ...) defines a temporary named result set. It improves readability for complex queries and enables recursion with WITH RECURSIVE. Some databases materialize CTEs, which can affect performance.

Q36. What is pagination in Spring Data JPA?

Answer: Pagination splits large result sets into pages. A repository method accepts a Pageable and returns a Page<T> that includes content, total count, total pages, and metadata. Example: Page<Transaction> findByAccountNumber(String accountNumber, Pageable pageable);. Always cap the page size to prevent unbounded result sets.

Q37. What is @Query in Spring Data JPA?

Answer: @Query lets you define custom JPQL or native SQL for a repository method.

```
@Query("SELECT o FROM Order o WHERE o.status = :status")
List<Order> findByStatus(@Param("status") String status);
```

Use native SQL when you need database-specific features like window functions, CTEs, or hints.

Q38. What is @Modifying?

Answer: @Modifying marks a @Query method that performs an UPDATE or DELETE. It requires an active transaction. Use `clearAutomatically = true` if you want the persistence context cleared afterward to avoid stale entities.

Q39. What is the Specification API?

Answer: It lets you build dynamic, type-safe predicates programmatically. Each filter is a `Specification` that returns a `Predicate`, and they can be combined with `and()` / `or()`. It is best for search screens with optional filters.

Q40. What are projections?

Answer: Projections return only selected fields instead of full entities. They reduce memory use, avoid lazy-loading issues, and improve serialization performance. Interface-based projections are called closed projections; class-based DTOs are open projections.

Q41. What is Query By Example?

Answer: QBE lets you pass a probe entity with non-null fields as search criteria. It is useful for simple exact-match searches but falls short for range queries, LIKE, nested conditions, and dynamic optional filters.

Q42. What is the difference between first-level and second-level cache?

Answer: The first-level cache is the persistence context; it caches entities within one `EntityManager` session. The second-level cache sits above the persistence context and caches entities across sessions using providers such as EhCache, Caffeine, or Redis. The second-level cache is best for reference data that changes rarely, such as country codes or product categories.

Q43. What is the query cache, and why can it be a trap?

Answer: The query cache stores the IDs returned by a query. The second time the query runs, Hibernate looks up the IDs and fetches the entities from the second-level cache. If the second-level cache is not configured, or if the underlying tables change frequently, the query cache adds overhead without benefit.

Q44. How do you optimize batch inserts with Spring Data JPA?

Answer: Set `spring.jpa.properties.hibernate.jdbc.batch_size` to 50 or 100, process data in chunks and call `saveAll(chunk)`, periodically flush and clear the persistence context, use a `SEQUENCE` ID strategy with a matching allocation size, and avoid `IDENTITY` for large batches. For very large volumes, consider `StatelessSession` or raw JDBC batching.

Q45. What is a schema migration tool?

Answer: Tools like Flyway and Liquibase manage database schema changes through versioned scripts. They make environments reproducible and avoid the risks of `ddl-auto = update` in production. Using `spring.jpa.hibernate.ddl-auto=update` in production can drop columns or alter data unexpectedly.

Q46. What is the difference between a view and a materialized view?

Answer: A view is a virtual query; it does not store data. A materialized view stores the result physically and must be refreshed. Materialized views are useful for expensive aggregations in reporting but add refresh complexity.

Q47. What is a covering index, and when does it help?

Answer: A covering index includes all columns a query needs. The database answers the query from the index alone, avoiding table lookups. It helps read-heavy queries with stable access patterns.

Q48. How do you detect whether a query uses an index?

Answer: Use the database execution plan. In MySQL use `EXPLAIN SELECT ...`, in PostgreSQL use `EXPLAIN ANALYZE SELECT ...`, in Oracle use `EXPLAIN PLAN FOR ...`, and in SQL Server use the execution plan viewer or `SET SHOWPLAN_ALL ON`. Look for `type=ALL` or `Seq Scan` as signs of a full table scan.

Q49. What is the difference between `save()` and `saveAndFlush()`?

Answer: `save()` schedules the entity for insertion/update and defers SQL until flush. `saveAndFlush()` forces an immediate flush. Use `saveAndFlush` sparingly, only when the next step in the same transaction needs the row visible in the database.

Q50. What are composite keys, and how do you map them?

Answer: A composite key uses multiple columns to uniquely identify a row. Use `@EmbeddedId`

with an `@Embeddable` class for clean mapping, or `@IdClass` when matching legacy schemas.

Q51. What is `@EnableJpaAuditing`?

Answer: It enables automatic population of audit fields such as `@CreatedDate`, `@LastModifiedDate`, `@CreatedBy`, and `@LastModifiedBy`. It removes audit boilerplate and supports compliance requirements.

Q52. What is Hibernate Envers?

Answer: Hibernate Envers provides entity versioning and audit history. It tracks every insert, update, and delete, storing snapshots in audit tables. It is useful for compliance and debugging data changes.

Q53. What is the difference between `getReferenceById` and `findById`?

Answer: `findById` loads the full entity immediately. `getReferenceById` returns a proxy without hitting the database. Use `getReferenceById` when you only need a foreign-key reference; use `findById` when you need the data.

Q54. How do you handle an `OptimisticLockException` in production?

Answer: For user-facing flows, return `409 Conflict` or retry a bounded number of times with backoff. For batch jobs, log the conflict and continue or move the record to a retry queue. For high-contention flows, consider redesigning with queues or pessimistic locking.

Q55. What is a deadlock, and how do you prevent it?

Answer: A deadlock occurs when two transactions each hold a lock and wait for a lock held by the other. Prevent it by always locking resources in a consistent order, such as by account ID ascending, or by using a single update statement where possible.

Q56. What is the purpose of `@Column`, `@Table`, and `@Entity`?

Answer: `@Entity` marks a class as mapped. `@Table` maps it to a specific table and schema. `@Column` maps a field to a specific column with constraints like name, nullable, and length. A new Java entity `Transaction` may map to a legacy Oracle table `TXN_HISTORY` with columns like `ACCT_NBR` and `TXN_AMT`.

Q57. What is `@Temporal`, and is it still relevant?

Answer: `@Temporal` maps `java.util.Date` or `Calendar` to a SQL date/time/timestamp. With Java 8+, prefer `java.time` types such as `LocalDate`, `LocalDateTime`, or `Instant`, which are type-safe and avoid timezone ambiguities.

Q58. What is a Hibernate proxy?

Answer: A Hibernate proxy is a runtime subclass created for lazy loading. It loads data only when a method is called. Because of this, `entity.getClass()` may return a proxy class name, and equality checks using `getClass()` can fail. Use `Hibernate.getClass(entity)` or `instanceof` in `equals()`.

Q59. What are the risks of returning JPA entities directly from a REST controller?

Answer: Serialization may trigger `LazyInitializationException`, bidirectional relationships can cause infinite recursion, internal fields leak to the API contract, and future schema changes break API consumers. The better approach is to return a DTO mapped inside the service layer.

Q60. What is the difference between `Page<T>` and `Slice<T>`?

Answer: `Page<T>` includes the total count of records, which requires an extra `COUNT` query. `Slice<T>` only knows whether there is a next page, so it is cheaper when the total count is not needed.

Scenario based questions

S1. A banking customer reports that a duplicate transaction was created during a fund transfer. How do you investigate and prevent this?

Answer: Duplicate transactions usually come from retry behavior on the client side, missing idempotency, or a race condition on the server. I start by checking the request logs for the same client request sent twice, looking at timestamps, correlation IDs, and business reference numbers. I query the transaction table for rows with the same reference or amount around the reported time. To prevent recurrence I introduce an idempotency layer: either store processed keys in a dedicated table with a unique constraint keyed by `idempotency_key` and `client_id`, or use a naturally unique business reference. The service should first check for an existing record before processing, and on a duplicate it returns the previously computed result. The operation must remain inside a transaction so the idempotency record and the business outcome are committed atomically.

S2. An API that worked in staging is slow in production. What do you check first?

Answer: The first step is to reproduce the slow call in production and capture the exact SQL being executed using `show-sql`, a query log, or an APM trace. Then I compare the production execution plan against staging with the same query shape. Production has larger data volumes, different data distributions, and possibly missing indexes. Common causes are N+1 selects from lazy collections, a missing index that existed in staging, stale table statistics leading to bad plans, lock contention, or a different Hibernate/datasource configuration. I also verify whether connection-pool settings, batch size, and cache configuration are identical. The fix depends on the root cause: add or rebuild the index, rewrite the query with `JOIN FETCH` or a projection, tune batching, or paginate large result sets.

S3. A report query times out after a schema change. What is your approach?

Answer: I run `EXPLAIN / EXPLAIN ANALYZE` on the report query against the new schema to see if the plan changed. The usual culprits are a dropped index, a changed column type that prevents index usage, stale statistics after the change, or the optimizer choosing a nested-loop join over a hash join because cardinality estimates are wrong. I also confirm the query still references the correct columns and that no implicit conversion was introduced. If the report is intrinsically expensive, I consider a materialized view, a pre-aggregated summary table, or moving the workload to a read replica. If the performance regression is blocking business users and the fix will take time, the safest step is to roll back the schema change while a permanent solution is designed and tested.

S4. After a deployment, orders and their items are out of sync — some items have no order. What happened?

Answer: Orphaned items usually mean the relationship was modified without maintaining both sides of a bidirectional mapping, or a cascade was misconfigured. For example, if code removes an item from `Order.items` but does not set `orderItem.order = null`, Hibernate may not recognize the change. Another cause is `CascadeType.REMOVE` or `orphanRemoval` set incorrectly on the wrong side. A bulk delete or a manual migration that bypassed JPA cascades can also leave child rows behind. To fix it I audit the relationship code, ensure helper methods update both sides, remove dangerous cascades, and add a database-level foreign key if one is missing. For existing orphans I run a cleanup script and add a not-null constraint on the foreign key to prevent recurrence.

S5. A nightly batch job starts failing with `OutOfMemoryError`. It processes millions of rows with `saveAll()`.

Answer: The persistence context accumulates every entity loaded or saved until the transaction commits, so a single transaction over millions of rows eventually exhausts heap memory. The fix is to process data in chunks, for example 500 or 1,000 records at a time, and after each chunk call

`entityManager.flush()` followed by `entityManager.clear()` to detach the processed entities. I also set `hibernate.jdbc.batch_size` to match the chunk size and use a SEQUENCE ID generator with a matching allocation size, because `IDENTITY` forces an immediate insert per entity and breaks batching. If the job is purely insert-only and extremely large, `StatelessSession` or raw JDBC batching is a better choice because it avoids the dirty-checking and event-listener overhead of the full persistence context.

S6. A dashboard endpoint causes `LazyInitializationException` only under load. Why?

Answer: Under normal load the controller serializes the response before the transaction-bound session closes, so lazy associations happen to work. Under load, thread scheduling, connection-pool wait times, or asynchronous serialization can delay JSON rendering until after the transaction and session have ended. The real fix is not to rely on timing. Load every lazy association inside the service transaction with `JOIN FETCH` or an entity graph, or return DTOs that contain only the data needed by the UI. Enabling Open Session in View (OSIV) is a workaround that keeps the session open during rendering, but it hides the underlying design problem and often leads to N+1 queries in production.

S7. Two concurrent fund transfers between the same accounts deadlock. How do you resolve it?

Answer: A deadlock happens when transaction A locks account 1 and waits for account 2, while transaction B locks account 2 and waits for account 1. I reproduce it with a test that runs two transfers in opposite directions concurrently. The permanent fix is to always acquire locks in a deterministic order, such as by account ID ascending, in every code path that updates multiple accounts. If locking is required, use `LockModeType.PESSIMISTIC_WRITE` on the accounts within a transaction. I also add a small, bounded retry with exponential backoff for legitimate lock waits, but the primary defense is consistent lock ordering. An alternative is to serialize transfers between the same accounts through a queue or a single account-pair mutex.

S8. A cache hit rate is high, but users still see stale data after an update. What could be wrong?

Answer: A high hit rate only means the cache is being read; it does not mean the data is correct. Stale data usually means the update path did not invalidate or refresh the cache entry. In a multi-instance deployment, each node may have a local cache that is not synchronized, so one instance updates the database while another continues serving old data from its local cache. Solutions include switching to a shared cache such as Redis, adding explicit cache invalidation in the write path, using cache eviction events, or choosing a shorter TTL if eventual staleness is acceptable. The cache key must also include all parameters that affect the result, otherwise an update in one dimension may leave stale entries for another dimension.

S9. A query returns different totals each time it is run during market hours. Why?

Answer: The query is running against a live transactional table while new rows are continuously inserted, committed, or rolled back. Aggregations over active tables are inherently non-repeatable unless the transaction runs in a snapshot isolation level or against a static snapshot. Solutions include reading from a read replica that reduces contention on the primary, using snapshot isolation if the database supports it, running the report against a materialized view or pre-aggregated table, or scheduling the report during a quiet window when write volume is low. For business-critical reports, an event-sourced or CDC-based reporting pipeline gives the most consistent results.

S10. A service throws `DataIntegrityViolationException` after a code change. How do you debug?

Answer: `DataIntegrityViolationException` is a Spring wrapper, so I read the nested root cause to identify the exact constraint that failed: a unique constraint, a not-null constraint, a check constraint, or a foreign-key violation. The name of the constraint usually points to the table and column. I then compare the failing request against the changed code to see whether it now inserts duplicates, omits a required field, passes a value that is too long, or references a parent row that no longer exists. The fix generally includes adding server-side validation before the database call and returning a clear domain-specific error to the caller instead of leaking the raw SQL exception.

S11. A new `@Transactional` service does not roll back on a custom checked exception. What is the fix?

Answer: By default Spring rolls back only on unchecked exceptions and `Error`, not on checked exceptions. The fix is to add `rollbackFor = MyException.class` to the `@Transactional` annotation, or `rollbackFor = Exception.class` if every exception should roll back. I also verify that the exception is not being caught and swallowed inside the method, because a swallowed exception never reaches the Spring proxy. Finally I confirm the method is `public` and is called through the Spring proxy; if it is `private`, `final`, or invoked from within the same bean, the annotation is ignored entirely.

S12. After switching databases, native queries fail. What do you check?

Answer: Native queries are tied to a specific database dialect, so moving from MySQL to PostgreSQL or Oracle often breaks function names, pagination syntax, string concatenation, date functions, and keywords like `LIMIT`, `ROWNUM`, or `FETCH FIRST`. I check each native query against the new dialect, starting with the ones in the error logs. The long-term fix is to use JPQL where possible and keep native SQL only for cases that require database-specific features. For those cases, I maintain per-dialect query versions or use a thin abstraction that selects the correct SQL based on the active dialect.

S13. A microservice loses updates during a database failover. How do you investigate?

Answer: First I check whether the application was writing while the primary database was failing over. If a transaction was in-flight and not yet acknowledged by the database, the write may or may not have been applied, leading to ambiguity. I verify the JDBC driver and connection-pool settings for failover and reconnect behavior, and I review application timeouts and retry logic. The safest architectural pattern for critical writes is to make them idempotent using a business key, and to use the outbox pattern so the event is persisted in the same transaction as the business update and published reliably later. That way a failover does not lose the intent of the write; replay simply reprocesses idempotent operations.

S14. A scheduled reconciliation job reports missing rows that the API claims were saved. What do you check?

Answer: I first confirm the API transaction was actually committed before the job started. If the job reads from a read replica, replication lag can make recently committed rows invisible. I also check whether the job and the API agree on status values, soft-delete flags, and date ranges. Timezone mismatches in date filters are a common cause: a row saved at 23:58 UTC on one day may be interpreted as saved on the next day depending on the time zone used by the job. If both services use an event log or CDC, I verify the consumer offset is not lagging behind the producer.

S15. A @Modifying update runs but the count seems too low. Why?

Answer: The update count can be lower than expected for several reasons. The `WHERE` clause may be stricter than intended, for example due to a wrong status filter or a date range that excludes rows. If the same entities were loaded into the current persistence context before the update, the later flush can overwrite the updated columns with the old in-memory values, effectively reverting the change. If the method is not inside a transaction, it may run in auto-commit mode and fail silently or partially. Finally, another transaction may have modified or deleted the rows between the read and the update, causing a lost update or lower match count. The fix is to validate the filter, clear the persistence context before the bulk update, ensure the method is `@Transactional`, and use optimistic or pessimistic locking if concurrent updates are possible.

S16. A developer adds FetchType.EAGER to every relationship to avoid lazy-loading errors. What do you say?

Answer: This is an anti-pattern. Eager loading pulls in data even when it is not needed, producing larger SQL joins, Cartesian products, and unpredictable memory and network usage. It does not scale as the object graph grows. The correct fix is to understand why the lazy load was needed outside the transaction: either load the association explicitly with `JOIN FETCH` or an entity graph inside the service, or return a DTO that contains only the data the caller needs. `FetchType.EAGER` should be reserved for small, tightly coupled relationships that are truly always required together.

S17. A migration script alters a column type in a table with hundreds of millions of rows. What concerns you?

Answer: Changing a column type on a huge table can take a long time, hold a metadata lock or table lock, fill transaction logs, and cause replication lag or application timeouts. Rollback is also slow and risky. In production I prefer online schema-change tools such as `pt-online-schema-change` for MySQL, `gh-ost`, or Oracle `DBMS_REDEFINITION`. A safer manual approach is backward-compatible: add a new column with the desired type, backfill it in small batches while the old column is still in use, switch the application to read and write the new column, and only then drop the old one. This avoids a long lock and gives a quick rollback path.

S18. A service method returns a JPA entity directly to a REST controller. What problems can arise?

Answer: Returning entities directly couples the API contract to the database schema. Serialization may trigger `LazyInitializationException` if associations were not loaded. Bidirectional relationships can cause infinite recursion during JSON serialization unless Jackson annotations are added. Internal fields such as IDs, versions, or audit columns leak into the API, and any future schema change can break API consumers. The better pattern is to map the entity to a DTO inside the service layer and return the DTO. This gives full control over the exposed fields, avoids lazy-loading problems, and makes versioning the API easier.

S19. Someone uses `@Transactional` on a private method. What do you tell the author?

Answer: It will not work. Spring transactional behavior is implemented through proxies, and proxies can only intercept `public` methods on Spring beans. `private`, `final`, `static`, and package-private methods are not intercepted. If the annotation is placed on a private helper, the code runs without a transaction and changes may commit or fail unexpectedly. The method should be made `public`, or the logic should be moved to a public method or another bean. If internal calls need transactionality, consider using `AopContext.currentProxy()` or restructuring the service.

S20. A repository method returns `Page<Order>` but the service ignores the total count. What could be improved?

Answer: `Page<T>` issues an extra `COUNT` query to compute total pages, which can be expensive on large tables. If the service does not use the total count, switching to `Slice<T>` removes the count query while still supporting pagination and sorting. If even the next-page check is not needed, a simple `List<T>` with `Pageable` is sufficient. Removing the count query reduces database load and simplifies the query plan, especially for complex filters.

S21. A @Query uses string concatenation to build a WHERE clause with user input. What is wrong?

Answer: String concatenation with user input is vulnerable to SQL injection. Even if the input comes from an internal system, it should never be embedded directly into SQL. The correct approach is to use named parameters (`:param`) or positional placeholders (`?1`) in the `@Query` annotation. For dynamic WHERE clauses, use the Criteria API or the Specification API, which build the query safely. If native SQL must be constructed dynamically, use a prepared-statement style approach and validate inputs against an allow-list of values, column names, and sort fields.

S22. A developer writes CascadeType.ALL on a @ManyToOne relationship. What is the danger?

Answer: Cascading should flow from the parent to the child, not from the child to the parent or to shared entities. Placing `CascadeType.ALL` on a `@ManyToOne` from `OrderItem` to `Product` means deleting an order line could cascade a delete to the shared product, corrupting the catalog. Similarly, persisting an order item could try to persist or merge an existing product unexpectedly. The cascade should be on the `Order -> OrderItem` side, and even then only the exact operations that match the business rule should be enabled. Shared reference entities should never be cascaded from transactional children.

S23. A service does entityManager.flush() after every save inside a loop. What is the impact?

Answer: Each flush forces Hibernate to synchronize the persistence context with the database, executing the pending SQL immediately. Doing this after every iteration defeats the purpose of batching and generates many database round trips, which increases latency and load. The correct pattern is to accumulate a chunk of entities, call `saveAll(chunk)`, then flush and clear the `EntityManager` once per chunk. Setting `hibernate.jdbc.batch_size` to the chunk size and using a SEQUENCE ID generator allows the driver to send multiple inserts in a single network round trip.

S24. You see @Cacheable on a method that returns a list of orders from today. What is the problem?

Answer: Orders from today change frequently, so the cached value becomes stale almost immediately and cache invalidation is constantly triggered. Caching is most valuable for slow-changing or expensive-to-compute data, such as reference data, product catalogs, or heavy reports. The cache key must also include every parameter that affects the result, including filters, date ranges, and page information. Otherwise an update in one dimension may leave stale entries for another. For dynamic operational data, query optimization, indexing, and read replicas are usually better tools than caching.

S25. A method annotated with `@Transactional` calls another `@Transactional` method in a different bean. How do the transactions interact?

Answer: With the default propagation `REQUIRED`, the second method joins the existing transaction. Both methods commit or roll back together. If the second method uses `REQUIRES_NEW`, Spring suspends the first transaction, starts a new physical transaction, executes the second method, commits or rolls it back independently, and then resumes the first transaction. This is useful when the second operation must commit regardless of the first operation's outcome, such as audit logging. It is important to understand that `REQUIRES_NEW` still uses the same database connection by default unless connection handling is configured otherwise, so it is not a general solution for concurrency.